■ MAKING REAL-LIFE JARVIS IN PYTHON ■

by **Pulse of AI**

EPISODE 15: Autonomous App & Game Generator • Complete Commented Source Code & Architecture

■■ CRITICAL WARNING & PREREQUISITE NOTICE:
JARVIS is engineered as a highly modular, multi-file software ecosystem. Episode 15 directly imports and communicates with modules developed in preceding episodes (hardware_control.py, software_control.py, and screen_vision.py). **You MUST watch all preceding episodes (Parts 1 to 14) and extract the required code from their pinned comments, otherwise JARVIS will fail with missing import errors and will NOT work!**

■ Complete Episode Roadmap & Video Archive (Clickable Shorts)

SERIES PART (1 - 7)	DIRECT SHORT LINK	SERIES PART (8 - 14)	DIRECT SHORT LINK
Part 1: Voice Setup & Foundation	Watch Short ■	Part 8: Smart Volume & Brightness	Watch Short ■
Part 2: Real-Time Audio Streaming	Watch Short ■	Part 9: Deep Hardware Telemetry	Watch Short ■
Part 3: Gemini Live Audio Bridge	Watch Short ■	Part 10: Screen Vision AI Engine	Watch Short ■
Part 4: Bilingual Voice Modulations	Watch Short ■	Part 11: Grounding & UI Clicker	Watch Short ■
Part 5: Local Desktop App Launcher	Watch Short ■	Part 12: Hands-Free Media Controller	Watch Short ■
Part 6: Process Termination Tools	Watch Short ■	Part 13: Full YouTube Hands-Free Control	Watch Short ■
Part 7: RAM Purge & Ghost Killer	Watch Short ■	Part 14: Autonomous YouTube Ad Skipper	Watch Short ■

Architecture Overview: Episode 15 introduces real-time voice-driven agentic software synthesis. Using Google's gemini-3.6-flash via the official google-genai SDK, JARVIS listens to natural voice instructions (e.g., *'JARVIS, mere liye modern multi-window style calculator banao'*), generates fully operational Tkinter/Pygame GUI scripts on the fly, cleans up formatting errors, saves the code into generated_tools/, and launches the application inside an isolated subprocess.

■ MODULE 1: code_generator.py

```
"""
code_generator.py
Dynamic GUI and Python Tool Generation Engine for JARVIS.
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import os
import re
import time
import subprocess
from google import genai
from google.genai import types
from dotenv import load_dotenv

load_dotenv()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

def generate_and_run_code(prompt: str) -> str:
    """Generates a complete Python GUI script using Gemini, fixes syntax bugs, and executes it."""
    system_prompt = (
        f"User request: '{prompt}'. "
        "Write a complete, fully functional Python script that includes a clean Graphical User Interface (GUI) "
        "using Python's built-in 'tkinter' library. Make it look modern, clean, and properly sized. "
        "IMPORTANT: In tkinter geometry specs like root.geometry('450x650'), NEVER put spaces around the 'x' (e.g., use '450x650', not '450 x 650'). "
        "Return ONLY valid Python code enclosed in ```python and ``` without any extra text or markdown descriptions."
    )

    # Retry loop to gracefully handle temporary 503 high-demand server spikes
    response = None
    for attempt in range(3):
        try:
            response = client.models.generate_content(
                model='gemini-3.6-flash',
                contents=system_prompt,
                config=types.GenerateContentConfig(temperature=0.2)
            )
            break
        except Exception:
            if attempt == 2:
                return "Failed to generate UI code due to high server demand, sir. Please try again."
            time.sleep(1.5)

    if not response or not response.text:
        return "Failed to receive generated code from the model, sir."

    try:
        code_text = response.text
        if "```python" in code_text:
            code_text = code_text.split("```python")[1].split("```")[0].strip()
        elif "```" in code_text:
            code_text = code_text.split("```")[1].split("```")[0].strip()

        # Fix any lingering spacing bugs in geometry specs (e.g. "450 x 650" -> "450x650")
        code_text = re.sub(r'(\d+)\s+x\s+(\d+)', r'\1x\2', code_text)

        os.makedirs("generated_tools", exist_ok=True)
        file_path = os.path.join("generated_tools", "dynamic_gui_tool.py")

        with open(file_path, "w", encoding="utf-8") as f:
            f.write(code_text)

        subprocess.Popen(["python", file_path])
        return f"Successfully generated and launched GUI application for: {prompt}, sir."
    except Exception as e:
        return f"Failed to parse and execute UI code, sir: {str(e)}"
```

■ MODULE 2: agent.py

```
"""
agent.py
Real-Time Gemini Live Engine with Semi-Transparent Cyberpunk HUD GUI
Voice + Text Input + Synced Speech Transcripts + Tool/UI Generator + Autonomous Ad Skipper (Episode 15)
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import math
import struct
import threading
import tkinter as tk
from tkinter import scrolledtext
import pyaudio
import winsound
import keyboard
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    restore_master_volume,
)

# Software, Media Telemetry & Navigation Tools
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
```

```
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision & Autonomous Grounding Tools
from screen_vision import (
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
)

# Dynamic Code & GUI Generation Tool
from code_generator import (
    generate_and_run_code,
    open_generated_tool,
)

load_dotenv()

console = Console()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
media_is_playing_state = False

# --- Semi-Transparent Cyberpunk Minimalist HUD ---
class JarvisHUD:
    def __init__(self, send_callback):
        self.send_callback = send_callback
        self.root = tk.Tk()
        self.root.title("■ JARVIS NEURAL HUD")
        self.root.geometry("640x500")
        self.root.configure(bg="#0B0F19")

        self.root.attributes("-topmost", True)
        self.root.attributes("-alpha", 0.88)
```

```
        header = tk.Label(
            self.root,
            text="JARVIS NEURAL COMMAND HUD",
            font=("Consolas", 12, "bold"),
            fg="#00E5FF",
            bg="#0B0F19"
        )
        header.pack(pady=(8, 2))

        sub_header = tk.Label(
            self.root,
            text="Autonomous Vision, UI Generator & Audio Engine • Pulse of AI",
            font=("Consolas", 8),
            fg="#70829B",
            bg="#0B0F19"
        )
        sub_header.pack(pady=(0, 6))

        self.chat_display = scrolledtext.ScrolledText(
            self.root,
            wrap=tk.WORD,
            font=("Consolas", 10),
            bg="#111827",
            fg="#E2E8F0",
            insertbackground="#00E5FF",
            relief=tk.FLAT,
            padx=10,
            pady=8
        )
        self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True)
        self.chat_display.config(state=tk.DISABLED)

        self.chat_display.tag_config("user", foreground="#38BDF8", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis", foreground="#4ADE80", font=("Consolas", 10))
        self.chat_display.tag_config("tool", foreground="#FBBF24", font=("Consolas", 9, "italic"))
        self.chat_display.tag_config("system", foreground="#94A3B8", font=("Consolas", 9))

        input_frame = tk.Frame(self.root, bg="#0B0F19")
        input_frame.pack(fill=tk.X, padx=12, pady=(4, 10))

        self.entry_box = tk.Entry(
            input_frame,
            font=("Consolas", 10),
            bg="#1E293B",
            fg="#F8FAFC",
```

```
            insertbackground="#00E5FF",
            relief=tk.FLAT
        )
        self.entry_box.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0, 6))
        self.entry_box.bind("<Return>", self.on_send)

        self.send_btn = tk.Button(
            input_frame,
            text="SEND",
            font=("Consolas", 9, "bold"),
            bg="#0284C7",
            fg="#FFFFFF",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=14,
            command=self.on_send
        )
        self.send_btn.pack(side=tk.RIGHT, ipady=4)

    def append_log(self, text: str, tag: str = "system"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, text + "\n", tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def on_send(self, event=None):
        msg = self.entry_box.get().strip()
        if msg:
            self.append_log(f"YOU: {msg}", "user")
            self.entry_box.delete(0, tk.END)
            self.send_callback(msg)

    def run(self):
        self.root.mainloop()

hud_instance = None

def start_gui(loop, text_queue):
    global hud_instance
    def on_user_submit(message: str):
        asyncio.run_coroutine_threadsafe(text_queue.put(message), loop)

    hud_instance = JarvisHUD(send_callback=on_user_submit)
    hud_instance.run()
```

```
def log_to_gui(text: str, tag: str = "system"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_log, text, tag)

def calculate_rms(data: bytes) -> float:
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

# Master Tools Manifest
tools = [
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
```

```
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
    generate_and_run_code,
    open_generated_tool,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]■■ [MEDIA PLAYING DETECTED] Push-to-Talk Armed: Hold [Right Ctrl] to speak.[/bold yellow]")
                    log_to_gui("[MEDIA DETECTED] Push-to-Talk Armed: Hold [Right Ctrl] or type below.", "system")
                else:
                    console.print("\n[bold green]■ [MEDIA PAUSED/IDLE] Hands-Free Mode Active. Speak freely.[/bold green]")
                    log_to_gui("[MEDIA IDLE] Hands-Free Voice Mode Active.", "system")
        except Exception:
            pass
        await asyncio.sleep(1.0)

async def handle_text_box_input(session, text_queue):
    while True:
        try:
            text = await text_queue.get()
            if text:
                console.print(f"[bold magenta]■■ [HUD INPUT][/bold magenta] {text}")
                await session.send_realtime_input(text=text)
        except Exception:
            await asyncio.sleep(0.05)

async def send_audio(session):
```

```
    global model_is_speaking, media_is_playing_state
    mic_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_IN,
        input=True,
        frames_per_buffer=CHUNK,
    )
    console.print("[cyan]■ [LISTENING] Dynamic Audio & Terminal Input Active...[/cyan]")

    silence_frame = b'\x00' * (CHUNK * 2)

    try:
        while True:
            data = await asyncio.to_thread(
                mic_stream.read, CHUNK, exception_on_overflow=False
            )

            if model_is_speaking:
                await session.send_realtime_input(
                    audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                )
                continue

            if media_is_playing_state:
                if keyboard.is_pressed("right ctrl"):
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )
            else:
                rms = calculate_rms(data)
                if rms > 450:
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )

    except asyncio.CancelledError:
        raise
```

```
    finally:
        try:
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

async def receive_audio(session, audio_queue):
    global model_is_speaking
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website, target_url)
                        elif call.name == "open_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(open_application, str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(close_application, str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await asyncio.to_thread(purge_background_tasks)
                        elif call.name == "control_media":
                            action = call.args.get("action", "play_pause")
                            result = await control_media(action)
                        elif call.name == "get_current_media_info":
                            result = await get_current_media_info()
                        elif call.name == "navigate_shorts":
                            direction = call.args.get("direction", "next")
```

```
                result = await asyncio.to_thread(navigate_shorts, direction)
            elif call.name == "skip_video":
                direction = call.args.get("direction", "next")
                result = await asyncio.to_thread(skip_video, direction)
            elif call.name == "seek_video":
                action = call.args.get("action", "forward")
                result = await asyncio.to_thread(seek_video, action)
            elif call.name == "toggle_video_view":
                mode = call.args.get("mode", "fullscreen")
                result = await asyncio.to_thread(toggle_video_view, mode)
            elif call.name == "set_volume":
                level = call.args.get("level", 70)
                result = await asyncio.to_thread(set_volume, level)
            elif call.name == "get_volume":
                result = await asyncio.to_thread(get_volume)
            elif call.name == "set_brightness":
                level = call.args.get("level", 50)
                result = await asyncio.to_thread(set_brightness, level)
            elif call.name == "get_brightness":
                result = await asyncio.to_thread(get_brightness)
            elif call.name == "get_battery_status":
                result = await asyncio.to_thread(get_battery_status)
            elif call.name == "get_system_vitals":
                result = await asyncio.to_thread(get_system_vitals)
            elif call.name == "get_system_temperatures":
                result = await asyncio.to_thread(get_system_temperatures)
            elif call.name == "analyze_screen":
                prompt = call.args.get("prompt", "Analyze what is visible on my screen.")
                result = await analyze_screen(prompt)
            elif call.name == "find_ui_element":
                element_name = call.args.get("element_name", "")
                result = await find_ui_element(element_name)
            elif call.name == "skip_youtube_ad":
                result = await skip_youtube_ad()
            elif call.name == "generate_and_run_code":
                prompt = call.args.get("prompt", "")
                result = await asyncio.to_thread(generate_and_run_code, prompt)
            elif call.name == "open_generated_tool":
                tool_name = call.args.get("tool_name", "")
                result = await asyncio.to_thread(open_generated_tool, tool_name)

            console.print(f"\n[bold yellow]■■ [TOOL EXECUTED: {call.name}][/bold yellow] [green]{result}[/green]")
            log_to_gui(f"■■ [{call.name}] {result}", "tool")

            function_responses.append(
                types.FunctionResponse(
```

```
                                name=call.name,
                                id=call.id,
                                response={"result": result},
                            )
                        )

                    if function_responses:
                        await session.send_tool_response(function_responses=function_responses)

                server_content = response.server_content
                if server_content is not None:
                    if server_content.interrupted:
                        while not audio_queue.empty():
                            audio_queue.get_nowait()
                        model_is_speaking = False

                    if server_content.input_transcription:
                        user_spoken = server_content.input_transcription.text
                        if user_spoken.strip():
                            console.print(f"\r[bold white][USER][/bold white] {user_spoken}")
                            log_to_gui(f"YOU (Voice): {user_spoken}", "user")

                    model_turn = server_content.model_turn
                    if model_turn is not None:
                        if not model_is_speaking:
                            model_is_speaking = True
                            console.print("[bold cyan]■ [JARVIS ANSWERING...][/bold cyan] ", end="")

                        full_speech_chunk = ""
                        for part in model_turn.parts:
                            if part.text:
                                console.print(f"[cyan]{part.text}[/cyan]", end="")
                                full_speech_chunk += part.text

                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)

                        if full_speech_chunk.strip():
                            log_to_gui(f"JARVIS: {full_speech_chunk.strip()}", "jarvis")

                    if server_content.turn_complete:
                        model_is_speaking = False
                        console.print("\n[cyan]■ [LISTENING] Awaiting instructions...[/cyan]")

    except asyncio.CancelledError:
        raise
```

```
    finally:
        await audio_queue.put(None)

async def play_audio(audio_queue):
    speaker_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_OUT,
        output=True,
        frames_per_buffer=CHUNK,
    )
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None:
                break
            await asyncio.to_thread(speaker_stream.write, audio_data)
    except asyncio.CancelledError:
        raise
    finally:
        try:
            speaker_stream.stop_stream()
            speaker_stream.close()
        except OSError:
            pass

async def run_session(config, model_id, greeting, text_queue):
    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✔ Secure Neural Bridge Established with Gemini Live.[/bold green]")
        log_to_gui("✔ Neural Bridge Established with Gemini Live.", "system")

        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        audio_queue = asyncio.Queue()
        monitor_task = asyncio.create_task(monitor_playback_state())
        send_task = asyncio.create_task(send_audio(session))
        text_task = asyncio.create_task(handle_text_box_input(session, text_queue))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))

        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, text_task, receive_task, playback_task)
```

```
        finally:
            monitor_task.cancel()
            send_task.cancel()
            text_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(monitor_task, send_task, text_task, receive_task, playback_task, return_exceptions=True)

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),
        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name="Puck"
                )
            )
        ),
        system_instruction=types.Content(
            parts=[
                types.Part.from_text(
                    text=(
                        "You are JARVIS, an advanced artificial intelligence assistant created by "
                        "Swayam Dhawale, Founder of Pulse of AI. "
                        "Accent and Persona: Speak in an articulate, natural, polite Indian English accent. "
                        "Calm, highly sophisticated, and articulate, exactly like Tony Stark's JARVIS. "
                        "Always address the user as 'sir'. "
                        "Tool Execution Rules: "
                        "1. When opening any website or URL, call open_website and say: 'Yes sir, opening [site name]'. "
                        "2. When opening desktop apps, games, or tools, call open_application and say: 'Yes sir, opening [app name]'. "
                        "3. When asked to close, shut down, terminate, or kill ANY app, browser, window, or game, call close_application with the 
                        "4. When asked to kill hidden background tasks, purge background processes, clean orphan threads, or optimize RAM, call pu
                        "5. When asked what song, video, or track is playing, or what media is active, call get_current_media_info and announce th
                        "6. When asked to pause, play, resume music or video, skip track, previous track, or mute/unmute audio, call control_media
```

```
                        "7. When asked to scroll shorts, go to next short, or next reel, call navigate_shorts with direction='next' and say: 'Yes
                        "8. When asked for next video, skip to next video in playlist, or next YouTube video, call skip_video with direction='next
                        "9. When asked to fast forward, skip 10 seconds ahead, or skip forward, call seek_video with action='forward'. When asked
                        "10. When asked to toggle full screen, enter full screen, or exit full screen, call toggle_video_view with mode='fullscree
                        "11. When setting volume, call set_volume and say: 'Yes sir, setting volume to [level]%'. "
                        "12. When asked what the volume level is, call get_volume and state the percentage crisply with 'sir'. "
                        "13. When setting brightness, call set_brightness and say: 'Yes sir, setting brightness to [level]%'. "
                        "14. When asked what the brightness is, call get_brightness and state the status crisply with 'sir'. "
                        "15. When asked about battery, power, or charging state, call get_battery_status and state the status with 'sir'. "
                        "16. When asked for temperatures, heat, thermal readings, or how hot the rig is running, call get_system_temperatures and
                        "17. When asked for full diagnostics, performance vitals, system status, or complete CPU/GPU/RAM stats, call get_system_vi
                        "18. When asked to inspect the screen, read an error, examine the terminal, or explain what is visible on display, call an
                        "19. When asked to locate or find a general UI icon, link, or window element, call find_ui_element. "
                        "20. MANDATORY AD SKIPPING RULE: Whenever the user says 'skip ad', 'skip this ad', 'skip video ad', 'click skip', or any v
                        "21. When asked to create a tool, build a UI application, design a calculator, or program a game window, call generate_and
                        "22. When asked to open, launch, or re-run a previously created tool, game, or script by name, call open_generated_tool wi
                        "General Rule: If the user asks a conversational question like 'can you hear me', simply respond politely with 'Yes sir, I
                        "Multilingual Rule: Support over 70 global languages. Detect the user's spoken language and respond fluently in that exact
                        "Respond in short, crisp sentences. Never speak markdown or bullet points. "
                        "If asked about your identity or creator, state clearly that you were built by Swayam Dhawale."
                    )
                )
            ]
        ),
    )

    console.print("[bold cyan]■■■
```